TTK4190 Assignment 2 Part 3
Path Following & State Estimation

Objective
In this assignment, you will test your heading and speed controllers by simulating a marine craft traveling between the Trondheim and Brekstad ferry terminals. The craft should follow straight-line paths defined by waypoints, as shown in Figure 1. To follow the path, you will implement a guidance system that computes a heading reference based on the ship's current position and the next waypoint (see Figure 2).

In problems 3 and 4, you will extend the simulator with state estimation. In practice, states like position, velocity, and orientation are often unavailable or noisy. To address this, you will implement a model-based Kalman filter to estimate the yaw angle and yaw rate and use these estimates in your heading controller instead of the true simulated states.

For this assignment, continue using your Matlab files from Assignment 2 Part 2.

On Canvas, you will find a file containing a set of five way-points, `WP.mat` to be used in this assignment. The first row of `WP.mat` contains the north coordinates, while the second row contains the east coordinates. The `pathplotter.m` file should be included in your Matlab directory and used to plot the ship's position.

Delivery Details
The tasks may be completed individually or in groups. The hand-in should be a PDF report and the code (.m files) for each sub-problem. Upload the .m files directly, not in a zip folder.

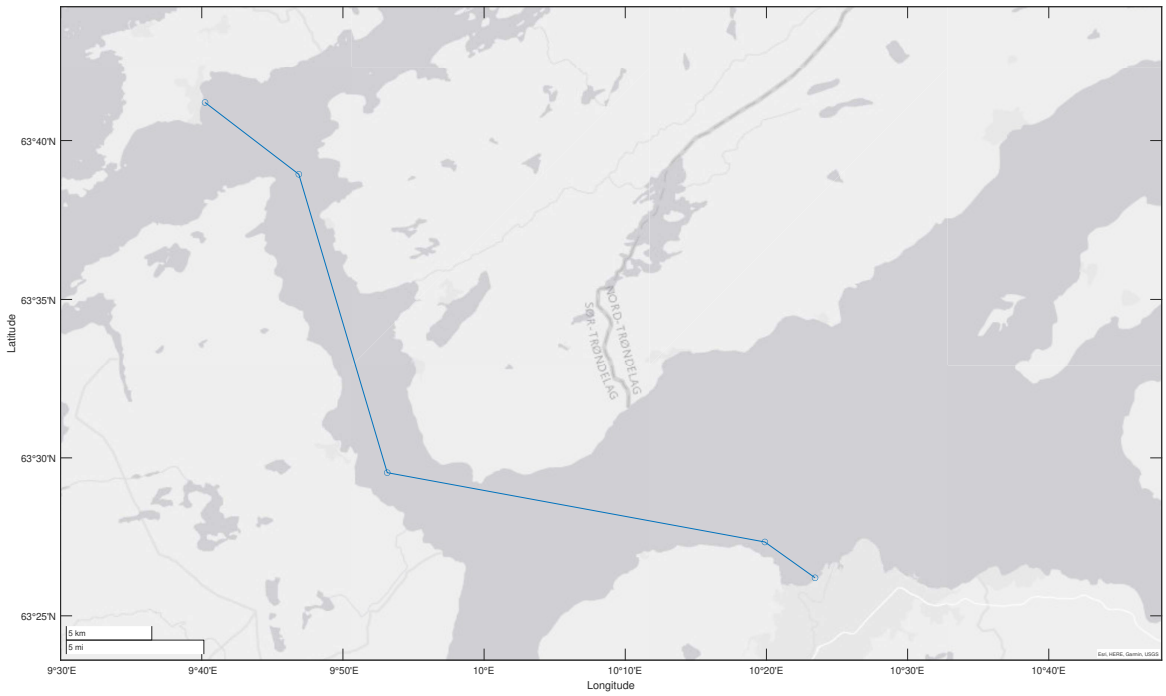


Figure 1: The objective is to get the marine craft from Trondheim to Brekstad ferry terminals by following the straight-line segments above using an appropriate guidance law.

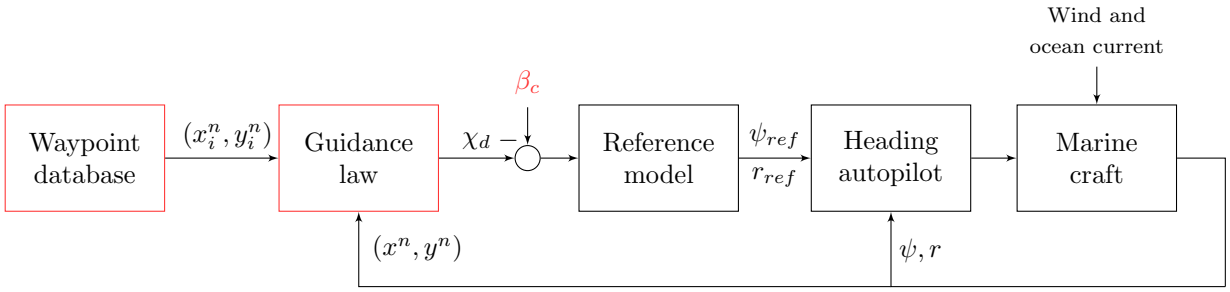


Figure 2: A block diagram illustrating the interaction between the way-points, guidance law, reference model, heading autopilot, and marine craft. The design and implementation of the way-points, guidance law and crab angle compensation (shown in red) are the main focus of this assignment.

For all problems in this assignment, set the initial heading angle to $-110°$ and use $U_{\text{ref}} = 9$ m/s.

Problem 1: LOS Guidance Law for Straight-Line Path Following

(a) Choose an appropriate guidance law and write a Matlab function called `guidance(.)`, which uses the ship position and path information as inputs and returns a time-varying reference signal for χ_d (desired course). Justify your choice of parameters in the guidance system and explain the principle you are using.
Hint:

$$\chi_d = \pi_h - \tan^{-1}(K_p y_e^p)$$

NB! This is Eq. (12.78) in (Fossen, 2021), but the notation has been updated from π_p to π_h for the new book in 2027.

(b) Modify your main code from the previous assignment to incorporate the guidance system and use the output as a reference signal for the reference model (i.e., $\psi_{\text{ref}} = \chi_d$). Simulate the system until you think the marine craft is sufficiently close to the final waypoint.

Utilize a modular approach during code development to make debugging easier. The resultant code should be similar in structure to the code below:

```
% Guidance law
[xk1,yk1,xk,yk,last] = WP_selector(x(4),x(5));
[e_y,pi_h] = crossTrackError(xk1,yk1,xk,yk,x(4),x(5));
chi_d = LOS_guidance(e_y,pi_h);
psi_ref = chi_d;
% Reference models
xd_dot = ref_model(xd,psi_ref);
```

Hint: If your ship spins in circles, see Figure 3, it's likely due to the reference model or your heading controller. First make sure the reference model state is initialized according to the true state. Then if you still struggle, try PD instead of PID control. Remember, you can adjust the simulation time by setting the number of samples `Ns`. Also, remember to include the function `pathplotter.m` in your Matlab folder and add the line `pathplotter(x,y)` at the end of `main.m`, after the other code used for plotting.

(c) Is the path-following behavior as expected? Does the current affect the results? Include relevant plots and explain why/why not.

Hint: Consider increasing the value of ω_{ref} in the reference model.

Problem 2: Crab Angle Compensation and Integral LOS

(a) For this simulation, turn the current off. Would you claim that crab angle compensation is necessary in this case? Include a plot showing the course χ, desired course χ_d, heading ψ, crab angle β_c, and sideslip β from the simulation results and comment on the results.

(b) For this simulation, turn the current on. Would you claim that crab angle compensation is necessary in this case? Include a plot showing the course χ, desired course χ_d, heading ψ, crab angle β_c and sideslip β from the simulation results and comment on the results.

(c) Design and implement a heading reference ψ_{ref} that takes into account the crab angle β_c. Simulate the marine craft again and comment on the differences between the results with and without crab angle compensation.

(d) Remove the crab angle compensation from (c) and modify your `guidance(.)` function to use the Integral LOS (ILOS) guidance law given by eqs. (12.108, 12.109) in (Fossen, 2021)

$$\psi_d = \pi_h - \tan^{-1}(K_p y_e^p + K_i y_{\text{int}}^p)$$
$$\dot{y}_{\text{int}} = \frac{\Delta y_e^p}{\Delta^2 + (y_e^p + \kappa y_{\text{int}}^p)^2}$$

where $K_p = 1/\Delta$, $K_i > 0$ and $\kappa > 0$ are design parameters. Simulate the marine craft again and comment on the results with ILOS. What are the pros and cons of using ILOS instead of direct crab angle compensation?

Problem 3: Kalman Filter Design

In problems 3 and 4 you will focus on adding state estimation to the simulation. The relevant framework blocks are highlighted in red in Figure 4.

(a) Design a model-based Kalman filter for estimating the yaw angle, yaw rate, and rudder bias. The rudder bias represents a slowly varying disturbance (caused by wind, ocean current, etc.) that should be subtracted from the rudder angle in the chosen model. The rudder bias can be modeled as a random-walk process driven by white noise. The yaw angle is the only measurement and the rudder angle is the only input. Write down the system equations (three differential equations) and the measurement model. White noise should be included where appropriate. Specify the Kalman filter matrices in continuous time (**A, B, C, D, E**), the state vector **x**, and the measurement y.

Hint: Use a linear model for the heading dynamics. You are not supposed to use the nonlinear equations of motion. Kalman filter design is described in (Chap. 13.4, Fossen, 2021).

(b) Discretize the continuous time system and write down the discrete-time matrices needed for implementation. First-order discretization is sufficient.

(c) Check the observability of the system. Can all the states be estimated?

Hint: You can use Matlab.

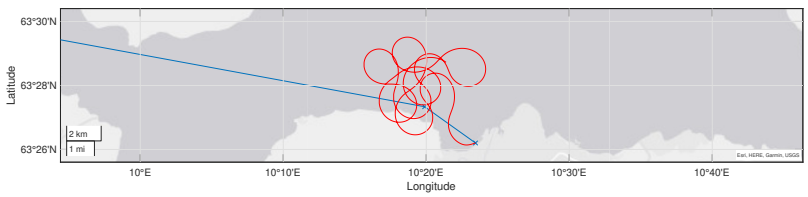


Figure 3: Unsuccessful path following due to poor tuning of the heading controller. Possible solutions: try using just PD or PID with anti-windup.

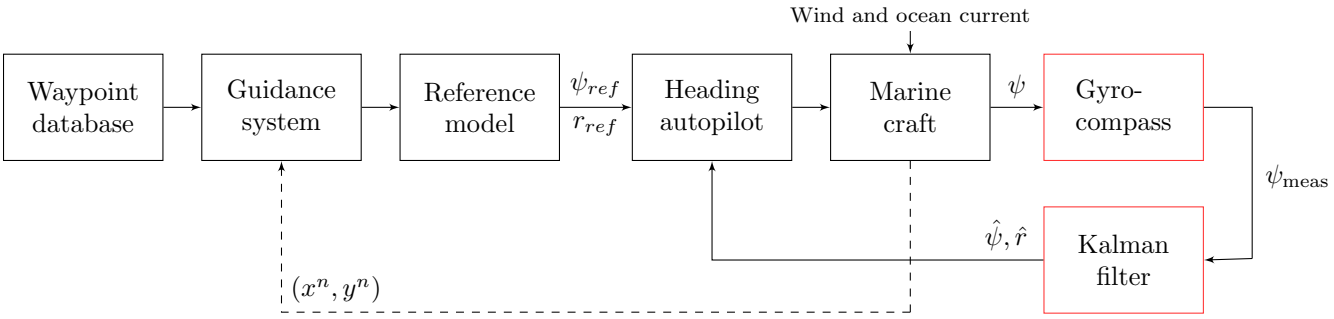


Figure 4: An overview of the guidance, navigation, and control blocks and how they interact. The navigation system (shown in red) comprises a gyrocompass and a Kalman filter and is the focus of this assignment.

Problem 4: Implementation

(a) Generate noisy measurements of the yaw angle and yaw rate. You are supposed to use the true states from the nonlinear model and add Gaussian distributed noise to these states. Use Gaussian noise with zero mean and a standard deviation of 0.5° for the yaw angle and 0.1° per second for the yaw rate. Plot the noisy measurements and compare them with the true states.

Hint: You can use the Matlab function `normrnd(.)` to generate Gaussian distributed noise.

(b) Implement the discrete-time Kalman filter in your code. It should be based on the matrices in Problem 3b) and the Kalman filter equations from Chapter 13.4.1 (Fossen, 2021). Only the noisy measurement of the yaw angle should be used in the Kalman filter together with the rudder input. Compare the state estimates with the true states (the true state of the rudder bias is not available, so it is sufficient to plot the estimate). Explain how you tuned $\mathbf{R}$, $\mathbf{Q}$ and $\mathbf{P}(0)$ in the Kalman filter. You can initialize the yaw angle, yaw rate, and rudder bias to zero.

The kalman filter should be implemented as a function your call from the main loop, it should look something like this:

```
% Run KF
[x_pst,P_pst,x_prd,P_prd] = KF(x_prd,P_prd,Ad,Bd,Ed,Cd,Qd,Rd,psi_meas,x(7));

% Redefine the errors based on the estimated states
e_psi = ssa(x_pst(1)-psi_d);
e_r = x_pst(2)-r_d;
e_u = x(1)-u_d;
```

Hint: Remember to redo the linearization from Part 2 Problem 2b) to find T and K constants of the first-order Nomoto-model linearized about the new desired surge speed $U_{\text{ref}} = 9$ m/s.

(c) Simulate the path following system (either using crab angle compensation or ILOS) using the noisy measurements of the yaw angle and yaw rate as feedback to the heading autopilot. How is the performance of the path-following system? Plot the position, desired rudder angle, yaw angle, and yaw rate, and comment on the results.

(d) Simulate the same path-following scenario using the state estimates as feedback to the heading autopilot. How is the performance of the path-following system? Plot the position, desired rudder angle, yaw angle, and yaw rate, and comment on the results.

Hint: You may have to adjust the controller gains (K_p, K_d, K_i) to get the desired behavior. The gains can be tuned by experimenting with various ω_b and ζ values. It may also be beneficial to drop the integral action in the heading autopilot (i.e., set $K_i = 0$).

Problem 5: Navigation Systems

This problem is intended to help place model-based navigation into a broader perspective. The answers should not be longer than 4-5 sentences.

(a) Describe shortly the main intention with a wave filter in model-based navigation systems.

(b) Describe shortly the main differences between model-based navigation systems and inertial navigation systems (INS). Specify advantages and disadvantages.

Problem 6: From Model-Based KF to Inertial State Estimation with ESKF

(a) The current implementation uses a model-based KF to estimate the yaw angle ψ, yaw rate r, and a rudder bias. The goal of this exercise is to replace the model-based KF with an inertial error-state Kalman filter (ESKF) using the `ins_euler` function from the MSS toolbox.

For this problem, start by listing the key differences between the model-based KF and the inertial ESKF in terms of:

- State vector and system model
- Sensor inputs and measurements
- Process and measurement noise

(b) You are now ready to implement the inertial ESKF using the `ins_euler` function from the MSS toolbox. Be aware that the `ins_euler` function is implemented for a 6-DoF system, whereas our ship simulator only models 3 degrees of freedom: motion in the xy-plane and rotation about the z-axis. The solution is to populate the unused states with zeros.

To help you structure your work, use the checklist below. You may refer to the example `SIMaidedINSeuler.m` for guidance.

- Define/initialize the ESKF state vector. It should include position, velocity, accelerometer biases, orientation, and gyroscope biases.
- Set up the covariance matrices `P_prd`, `Q_d`, and `R_d`. You may start with the values proposed in `SIMaidedINSeuler.m`.

```
P_prd = eye(15);
% Process noise weights: vel, acc_bias, w_nb, ars_bias
Qd = diag([0.1 0.1 0 0.001 0.001 0 0 0 0.1 0 0 0.001]);
% Measuremenet noise weights: pos, acc, compass
Rd = diag([0.1 0.1 0.1  1 1 1  0.01]);
```

Note: The measurement noise `R_d` corresponds to the three aiding sources used by `ins_euler`: GNSS position, a gravity reference for tilt correction, and the compass. The gravity reference is derived from the accelerometer measurement under the assumption of low linear acceleration.

- Generate the IMU measurements. Compute the true specific force and extract the true angular rate from the ship model. Add Gaussian noise and bias to these signals. You can choose the complexity of the bias yourself; the simplest option is to set it as a small constant. *Note:* You may start with the biases turned off for tuning and debugging.
- Generate aiding measurements: compass heading and GNSS position. You may use the following sensor noise specifications (specified as standard deviations).

```
% Sensor noise
sigma_accel = 0.001;              % m/s^2 (accelerometer)
sigma_gyro = 0.0000175;           % rad/s (gyroscope)
sigma_gps_pos = 0.025;            % meters (RTK GPS position)
sigma_gps_vel = 0.02;             % m/s (RTK GPS velocity)
sigma_compass_psi = deg2rad(0.5); % rad (compass)
```

- Implement the ESKF update using the `ins_euler` function. Ensure that aiding measurements are applied at the correct rate. Use a 10 Hz sampling rate for the IMU measurements (same as the simulation loop time step) and a 5 Hz sampling rate for the aiding GNSS data. *These values are not realistic; typically, you would have around 100 Hz for the IMU and 10 Hz for GNSS. However, with the provided values, the simulations take less time :)* Minimal example

based on `SIMaidedINSeuler.m`:

```
% Positions measurements are slower than the sampling time
if t(i) > t_slow
    % Position aiding + compass aiding
    [x_ins,P_prd] = ins_euler(x_ins,P_prd,mu,h,Qd,Rd,f_imu,w_imu,y_psi,y_pos
        );
    % Update the time for the next slow position measurement
    t_slow = t_slow + h_pos;
else
    [x_ins,P_prd] = ins_euler(x_ins,P_prd,mu,h,Qd,Rd,f_imu,w_imu,y_psi);
end
```

- Extract and store the relevant estimated states from the ESKF output for later comparison.

```
% Save simdata to simdata_ESKF.mat
save('simdata_ESKF.mat','simdata')
```

(c) Assess the performance of the autopilot when using the ESKF for state estimation. You may need to retune the heading PID gains to account for the new closed-loop dynamics. Include plots comparing desired values, actual values, and estimated values for key signals like yaw angle and yaw rate.

(d) Based on the simulation results, how does the selection of the state estimation filter affect the ship's autopilot performance? To what degree are the results explained by the fact that the filters are tested in a simulation? Do you expect the same relative performance in a real-world implementation?

Answers

4b) $T = -99.4713$, $K = -0.0049$ for $U_{\text{ref}} = 9$ m/s.